

Phase 2 patch — additions for Phase 3 hand-off

Apply each section verbatim to `phase2_optimiser.py`. Nothing outside the specified locations changes.

(1) Extend `PassiveFitResult`

Append the following blocks to the `PassiveFitResult` dataclass, after `n_validation_bundles: int = 0`:

python

```
# ----- Residual noise at MLE (for parametric bootstrap) -----
# Pooled std and lag-1 autocorrelation of (data - model) residuals
# at the MLE, computed over a pre-stim baseline window and the
# post-stim training window. Used by Phase 3 v2's bootstrap.
residuals_sigma_mV: float = float("nan")
residuals_rho_lag1: float = float("nan")

# ===== Phase-3 hand-off =====
# Three live references kept *only* on the success path of fit_one_cell.
# NOT safe to pickle: neuron_cell holds NEURON Hoc handles (process-local);
# gp_result is large; opt_inputs is fine but cleared for symmetry.
# fit_cells / _fit_worker leave all three as None; use fit_one_cell or
# rebuild_neuron_cells_for_phase3 to restore neuron_cell in the parent.
gp_result: Optional[Any] = field(default=None, repr=False)
neuron_cell: Optional[Any] = field(default=None, repr=False)
opt_inputs: Optional[Any] = field(default=None, repr=False)
```

(2) Populate them at the success-path return of `fit_one_cell`

Add the `_estimate_residual_noise_at_mle` call block (see section 6c) just before the final `return PassiveFitResult(...)`, then extend that return statement with the five new keyword arguments:

python

```

return PassiveFitResult(
    specimen_id=sid, layer=layer, dendrite_type=dt_type, F=F,
    fit_target=fit_target,
    cm_uF_per_cm2=cm_opt, rm_0hm_cm2=rm_opt, ra_0hm_cm=ra_opt,
    cm_sigma=sigmas["cm_sigma"],
    rm_sigma=sigmas["rm_sigma"],
    ra_sigma=sigmas["ra_sigma"],
    train_rmsd_mV=train_rmsd,
    valid_rmsd_mV=valid_rmsd,
    valid_to_train_ratio=ratio,
    rin_MOhm_allen=opt_in.rin_MOhm,
    tau_ms_allen=opt_in.tau_ms,
    v_rest_mV=opt_in.v_rest_mV,
    validation_status=status,
    n_calls=n_calls, n_initial=n_initial,
    wall_time_s=time.perf_counter() - t_start,
    error_message="",
    loss_history=loss_history,
    noise_sigma_mV=noise_sigma_mV,
    noise_rho_lag1=noise_rho_lag1,
    noise_rho_lag1_high=noise_rho_high,
    valid_rmsd_per_bundle=valid_per_bundle,
    n_validation_bundles=len(valid_per_bundle),
    residuals_sigma_mV=residuals_sigma_mV,
    residuals_rho_lag1=residuals_rho_lag1,
    # ----- Phase-3 hand-off -----
    gp_result=result,
    neuron_cell=cell,
    opt_inputs=opt_in,
)

```

(3) Strip live references from the DataFrame export inside `fit_cells`

In the `fit_cells` tidy-DataFrame loop, replace:

```
python
```

```
import dataclasses
r_export = dataclasses.replace(r, neuron_cell=None)
d = asdict(r_export)
d.pop("loss_history", None)
d.pop("valid_rmsd_per_bundle", None)
```

with:

python

```
import dataclasses
r_export = dataclasses.replace(r,
                                neuron_cell=None,
                                gp_result=None,
                                opt_inputs=None)

d = asdict(r_export)
d.pop("loss_history", None)
d.pop("valid_rmsd_per_bundle", None)
```

`dataclasses.replace` creates a shallow copy and nulls the three live-object fields **only on the export copy** — the originals in `results` are untouched and retain their live references for Phase 3.

(4) Clear `neuron_cell` in `_fit_worker` before returning

NEURON Hoc handles cannot cross process boundaries. `gp_result` and `opt_inputs` are pickleable and must be preserved for `rebuild_neuron_cells_for_phase3`.

Replace the existing `_fit_worker` body with:

python

```
def _fit_worker(args) -> PassiveFitResult:
    """Worker for multiprocessing.Pool.

    Rebuilds its own NEURON cell from the SWC path (NEURON sections are
    process-global and cannot be pickled).  gp_result and opt_inputs are
    preserved across the process boundary; neuron_cell is cleared to None.
    """
    (cell_data, opt_in, F, kwargs) = args
    from phase1_data_loader import build_neuron_model
    cell = build_neuron_model(cell_data.swc_path, F=F)
    r = fit_one_cell(cell, cell_data, opt_in, F=F, **kwargs)
    # Drop the un-picklable NEURON cell; gp_result and opt_inputs survive.
    r.neuron_cell = None
    return r
```

(5) Add `rebuild_neuron_cells_for_phase3` after `fit_cells`

Insert immediately after the closing of `fit_cells`, before `aggregate_population`:

```
python
```

```
def rebuild_neuron_cells_for_phase3(
    results: List[PassiveFitResult],
    cells_data: Sequence["CellData"],
    F: float,
    *,
    statuses: Sequence[str] = ("good", "to_refine"),
    verbose: bool = True,
) -> None:
    """Rebuild PassiveCell objects in the parent process after fit_cells().
```

When fit_cells() runs with n_workers > 1, neuron_cell comes back as None (NEURON Hoc handles cannot cross process boundaries). gp_result and opt_inputs ARE populated. This function fills in the missing neuron_cell so that Phase 3 can call the real NEURON simulator for CI validation.

Modifies results in-place.

```
"""
```

```
from phase1_data_loader import build_neuron_model
```

```
sid_to_cd = {int(cd.specimen_id): cd for cd in cells_data}
n_rebuilt = 0
```

```
for r in results:
    if r.validation_status not in statuses:
        continue
    if r.gp_result is None:
        continue # fit failed before GP was trained

    cd = sid_to_cd.get(int(r.specimen_id))
    if cd is None:
        warnings.warn(
            f"[rebuild_neuron_cells_for_phase3] specimen {r.specimen_id} "
            f"not found in cells_data - skipping."
        )
        continue

    try:
        cell = build_neuron_model(cd.swc_path, F=float(r.F))
        cell.set_passive(r.cm_uF_per_cm2, r.rm_0hm_cm2, r.ra_0hm_cm)
        cell.set_e_pas(r.v_rest_mV)
        r.neuron_cell = cell
        n_rebuilt += 1
    except Exception as e:
```

```
warnings.warn(
    f"[rebuild_neuron_cells_for_phase3] failed for specimen "
    f"{r.specimen_id}: {type(e).__name__}: {e}"
)

if verbose:
    print(f"[rebuild_neuron_cells_for_phase3] rebuilt {n_rebuilt} "
          f"NEURON model(s) out of {len(results)} result(s).")
```

(6) Residual noise estimation for Phase 3 v2 parametric bootstrap

6a — Two fields already added in section (1) above

`residuals_sigma_mV: float = float("nan")` `residuals_rho_lag1: float = float("nan")`

6b — Add the estimator helper in Cell 5 (before `_crop_window`)

python

```

def _estimate_residual_noise_at_mle(
    cell,
    train_bundles,
    v_rest_mV: float,
    cm_mle: float,
    rm_mle: float,
    ra_mle: float,
    train_window_ms: Sequence[float],
    pre_stim_window_ms: Optional[Sequence[float]] = None,
) -> Tuple[float, float]:
    """Pool (data - model) residuals at the MLE and return ( $\sigma$ ,  $\rho_{lag1}$ ).

    Residuals are drawn from two windows of each training bundle:
    - pre-stim baseline (pure noise)
    - post-stim training window (noise + model misspecification)

    The pooled  $\sigma$  is used as the noise scale for Phase 3's parametric
    residual bootstrap; it is more conservative than the pre-stim  $\sigma$  alone
    because it absorbs irreducible model-data mismatch.
    """
    cell.set_passive(cm_mle, rm_mle, ra_mle)
    cell.set_e_pas(v_rest_mV)

    train_t0_s, train_t1_s = (np.asarray(train_window_ms) * 1e-3).tolist()

    if pre_stim_window_ms is None:
        pre_t0_s = (train_window_ms[0] - 50.0) * 1e-3
        pre_t1_s = (train_window_ms[0] - 1.0) * 1e-3
    else:
        pre_t0_s, pre_t1_s = (np.asarray(pre_stim_window_ms) * 1e-3).tolist()

    pooled: List[float] = []
    for b in train_bundles:
        # Simulate using the same API as _build_loss_function / _simulate_square_sub
        t_sim_s, v_sim_raw = _simulate_square_subthreshold(cell, b, v_rest_mV)
        sim_v = _interp_to_grid(t_sim_s, v_sim_raw, b.t)

        pre_mask = (b.t >= pre_t0_s) & (b.t <= pre_t1_s)
        post_mask = (b.t >= train_t0_s) & (b.t <= train_t1_s)
        if (not pre_mask.any()) or (not post_mask.any()):
            continue

        baseline_data = float(b.v_mV[pre_mask].mean())

```

```

baseline_sim = float(sim_v[pre_mask].mean())

# Post-stim residuals (signal window: noise + model misspecification)
train_res = ((b.v_mV[post_mask] - baseline_data) -
              (sim_v[post_mask] - baseline_sim))
# Pre-stim residuals (pure noise around zero)
pre_res = b.v_mV[pre_mask] - baseline_data

pooled.extend(pre_res.tolist())
pooled.extend(train_res.tolist())

pool = np.asarray(pooled, dtype=float)
if pool.size < 3:
    return float("nan"), float("nan")

sigma = float(np.std(pool, ddof=1))
c      = pool - pool.mean()
rho    = float(np.sum(c[:-1] * c[1:]) / (np.sum(c**2) + 1e-12))
return sigma, rho

```

Critical note on `b.v_mV` vs `b.v_mean`: `SweepBundle` from Phase 1 stores the averaged voltage trace as `v_mV` (the same field that `_build_loss_function` uses). There is no `v_mean` attribute. Any occurrence of `b.v_mean` in this function is a bug and must be replaced with `b.v_mV`.

6c — Call it at the end of `fit_one_cell`, just before the final return

python

```

# Residual noise at MLE for Phase 3 v2 parametric bootstrap
try:
    residuals_sigma_mV, residuals_rho_lag1 = _estimate_residual_noise_at_mle(
        cell=cell,
        train_bundles=opt_in.train_bundles,
        v_rest_mV=opt_in.v_rest_mV,
        cm_mle=cm_opt, rm_mle=rm_opt, ra_mle=ra_opt,
        train_window_ms=opt_in.train_window_ms,
        pre_stim_window_ms=getattr(opt_in, "pre_stim_window_ms", None),
    )
except Exception as _e:
    warnings.warn(f"[fit_one_cell] residual-noise estimation failed: {_e}")
    residuals_sigma_mV, residuals_rho_lag1 = float("nan"), float("nan")

```

Then pass both scalars in the `PassiveFitResult(...)` constructor (section 2 above). They are plain floats and travel safely across the `multiprocessing.spawn` boundary.

Summary of all changes

Section	What changes	Why
(1)	Add <code>residuals_sigma_mV</code> , <code>residuals_rho_lag1</code> , <code>gp_result</code> , <code>neuron_cell</code> , <code>opt_inputs</code> to <code>PassiveFitResult</code>	Phase 3 needs noise estimates and live references
(2)	Populate all five new fields in <code>fit_one_cell</code> success-path return	Single-cell and sequential paths
(3)	Use <code>dataclasses.replace</code> to null live references before <code>asdict</code> in <code>fit_cells</code> DataFrame export	Prevents deepcopy crash on NEURON / sklearn objects
(4)	Clear <code>neuron_cell = None</code> in <code>_fit_worker</code> before return	NEURON Hoc handles not picklable; <code>gp_result</code> and <code>opt_inputs</code> are preserved
(5)	Add <code>rebuild_neuron_cells_for_phase3</code>	Parent process rebuilds NEURON models cheaply from SWC after HPC parallel run
(6)	Add <code>_estimate_residual_noise_at_mle</code> helper and call it in <code>fit_one_cell</code>	Computes σ_{residual} / ρ_{residual} at MLE for Phase 3 bootstrap noise model

Critical attribute name

`SweepBundle.v_mV` is the voltage array used throughout Phase 2. It does **not** have a `v_mean` or `label` attribute. Any code that references `b.v_mean` or `b.label` will raise `AttributeError` at runtime and must be corrected to `b.v_mV` and an integer index respectively.